

Guide for WRF simulation

Convention : one letter per run so nothing gets overwritten. Here the new run is fiona_wrf_pinn15g and the previous one, that everything is copied from, is fiona_wrf_pinn15f. I use \$NEW and \$OLD for the two paths in /scratch/gpfs/RVNE/gabrielm/, they reset between sessions and are not inherited inside a salloc, so they have to be exported again each time.

1. [Locally] :

use build_npz_hybrid.py (which is in my home in tiger3, and on the github [to do]) to create a .npz file like wps_input_hybrid15_g.npz that contains all the input information that we are ultimately giving to the WRF simulation. To read the PINN it uses the specification of the PINN that we are using. If the PINN changes any major parameter (does not concerns the training part), the same modifications should be made to build_npz_hybrid (I know this is not practical but I never changed anything concerning the PINN (the map (t,x,y,p)->(u,v,\omega, \Phi, T, q)). It takes the GFS data (for the soil part and skin temperature), and the HAFS data used. The blending is done here. It takes like 15minutes to run (on my computer) (might be less on a cluster). Only the first record is hybrid, the 7 others are pure HAFS since they only serve to build the boundary conditions.

The file is around 1Go (note that it can take a lot of time (30minutes) to transfer this file from your PC if you're at home using the VPN).

2. On tiger3, create \$NEW/{data,WPS,WRF} and put the npz in data/.	<pre>mkdir -p \$NEW/{data,WPS,WRF} scp -C -o MACs=hmac-sha2-256 "C:\Users\melka\Downloads\WRF\data\wps_input_hybrid15_g.npz" gm7719@tiger3.princeton.edu:/scratch/gpfs/RVNE/gabrielm/fiona_ wrf_pinn15g/data/</pre>
3. Fill \$NEW/WPS from the old run : copy geo_em.d01.nc, geo_em.d02.nc and namelist.wps, and symlink metgrid.exe from Hunter's build (/scratch/gpfs/GEOCLIM/hc8402/WRF/WPS/). Also symlink his metgrid folder itself, it is the one holding METGRID.TBL and metgrid.exe looks for it in relative path. That is why the same names appear both in my directory and in his. No geogrid to rerun, the domains never change. Copy make_intermediate.py into \$NEW/data/ and env.sh into \$NEW/.	<pre>cd \$NEW/WPS cp \$OLD/WPS/geo_em.d01.nc \$OLD/WPS/geo_em.d02.nc \$OLD/WPS/namelist.wps . ln -sf /scratch/gpfs/GEOCLIM/hc8402/WRF/WPS/metgrid.exe . ln -sf /scratch/gpfs/GEOCLIM/hc8402/WRF/WPS/metgrid ./metgrid cp \$OLD/data/make_intermediate.py \$NEW/data/ cp \$OLD/env.sh \$NEW/</pre>
4. In make_intermediate.py, one sed to replace the old run name by the new one and the old npz name by the new one, then grep to check it (don't run the sed twice). Also grep namelist.wps : should be 15 UTC the 20th to 12 UTC the 21st, interval_seconds = 10800, max_dom = 2.	<pre>sed -i "s fiona_wrf_pinn15f fiona_wrf_pinn15g g; s wps_input_hybrid15_f_new.npz wps_input_hybrid15_g.npz " \$NEW/data/make_intermediate.py grep -nE "npz fiona_wrf" \$NEW/data/make_intermediate.py grep -E "start_date end_date interval_seconds max_dom" \$NEW/WPS/namelist.wps</pre>

5. Take an allocation, <code>salloc --nodes=1 --ntasks=8 --time=00:40:00</code> . Never paste a multi-line block right after <code>salloc</code> , otherwise it fails	<code>salloc --nodes=1 --ntasks=8 --time=00:40:00</code>
6. Re-export <code>\$NEW</code> , then source <code>\$NEW/env.sh</code> (intel 2024.2 stack, because it has to match what Hunter compiled with).	<code>export NEW=/scratch/gpfs/RVNE/gabrielm/fiona_wrf_pinn15g</code> <code>source \$NEW/env.sh</code>
7. <code>ldd \$NEW/WPS/metgrid.exe grep -i "not found"</code> : nothing should come out. <code>/projects</code> is not mounted on the compute nodes while <code>env.sh</code> points <code>LD_LIBRARY_PATH</code> there, so this is where you catch it, and not in the middle of <code>metgrid</code> .	<code>ldd \$NEW/WPS/metgrid.exe grep -i "not found"</code>
8. module load <code>anaconda3/2024.6</code> then source <code>~/bashrc</code> then <code>conda activate wps_prep</code> (conda is not there by default on the nodes, this has to be redone on every allocation). <code>wps_prep</code> is python 3.11 + numpy + pywintner by pip. Pywintner cannot be installed on Windows, so I had to do this part in the cluster.	<code>module load anaconda3/2024.6</code> <code>source ~/bashrc</code> <code>conda activate wps_prep</code>
9. <code>python make_intermediate.py</code> , it writes the intermediate files in <code>\$NEW/WPS</code> , there should be 8 of them. Then, and this one is not optional, make the short name symlinks <code>pywintner</code> writes <code>HAFS:2022-09-20_15:00:00</code> while <code>metgrid</code> looks for <code>HAFS:2022-09-20_15</code> without the seconds, so without that loop <code>metgrid</code> finds nothing and stops.	<code>cd \$NEW/data && python make_intermediate.py</code> <code>cd \$NEW/WPS && ls HAFS:2022-09-2*:00:00 wc -l</code> <code>for f in HAFS:*_?:00:00; do ln -sf "\$f" "\${f%:00:00}"; done</code>
10. <code>conda deactivate</code> , then <code>./metgrid.exe</code> . You should get 16 <code>met_em</code> files (8 dates on 2 domains).	<code>conda deactivate</code> <code>./metgrid.exe</code> <code>ls met_em.d0?.2022-09-2*.nc wc -l</code>
11. In <code>/WRF</code> , symlink Hunter's <code>real.exe</code> and <code>wrf.exe</code> from <code>WRFV4.6.1/main/</code> , and from <code>WRFV4.6.1/run/</code> the <code>*.TBL</code> , <code>RRTMG*</code> , <code>ozone*</code> and <code>CAMtr_volume_mixing_ratio*</code> . That last one is the link that was missing once and killed <code>wrf.exe</code> at startup with a useless message. Don't replace all this by a single <code>ln -sf ../run/*</code> , it drags namelists you don't want. Symlink the <code>met_em</code> files from <code>/WPS</code> too.	[DO NOT RUN the ln simultaneously (you have to do each one by one), otherwise it breaks and its a mess] <code>cd \$NEW/WRF</code> <code>ln -sf /scratch/gpfs/GEOCLIM/hc8402/WRF/WRFV4.6.1/main/real.exe .</code> <code>ln -sf /scratch/gpfs/GEOCLIM/hc8402/WRF/WRFV4.6.1/main/wrf.exe .</code> <code>ln -sf /scratch/gpfs/GEOCLIM/hc8402/WRF/WRFV4.6.1/run/*.TBL .</code> <code>ln -sf /scratch/gpfs/GEOCLIM/hc8402/WRF/WRFV4.6.1/run/RRTMG* .</code> <code>ln -sf /scratch/gpfs/GEOCLIM/hc8402/WRF/WRFV4.6.1/run/ozone* .</code> <code>ln -sf /scratch/gpfs/GEOCLIM/hc8402/WRF/WRFV4.6.1/run/CAMtr_volume_mixing_ratio* .</code> <code>ln -sf \$NEW/WPS/met_em.d0?.2022-09-2*.nc .</code>
12. Copy <code>namelist.input</code> from the old run. It is the WRF configuration itself (time_control, domains, physics, dynamics), and we copy it unchanged on purpose : since we want to isolate the effect of the initialization,	<code>cp \$OLD/WRF/namelist.input .</code> <code>grep -E "run_hours start_hour end_hour num_metgrid_levels" namelist.input</code>

everything here has to stay identical between the HAFS baseline and the PINN runs. (Then it might be good to change some parameters to have some real improvements). You should have num_metgrid_levels = 46, it must match exactly what metgrid produced. A grep should show run_hours = 21 and start_hour = 15, 15.	
13. ulimit -s unlimited then srun -n 8 ./real.exe. The ulimit is essential, it segfaults without. tail -5 rsl.error.0000 should say SUCCESS COMPLETE REAL_EM INIT, and wrfinput_d01 / wrfinput_d02 / wrfbdy_d01 are around 122, 221 and 77 Mo. Then leave the allocation.	ulimit -s unlimited srun -n 8 ./real.exe tail -5 rsl.error.0000 ls -lh wrfinput_d01 wrfinput_d02 wrfbdy_d01 exit
14. Copy wrf.slurm from the old run and sed the run name inside it. The cd in the script is an absolute path, so if you forget that sed the job runs in the previous directory and overwrites its outputs. The same module list as env.sh, ulimit -s unlimited and srun ./wrf.exe.	cd \$NEW/WRF cp \$OLD/WRF/wrf.slurm . sed -i "s fiona_wrf_pinn15f fiona_wrf_pinn15g g; sljob-name=.* job-name=fionaP15g " wrf.slurm grep -nE "cd job-name nodes ntasks time=" wrf.slurm
15. sbatch wrf.slurm ; takes some time to get started but should take around 1h30 with my configuration. At the end, SUCCESS COMPLETE WRF in rsl.error.0000, 22 wrfout_d01 and 43 wrfout_d02.	sbatch wrf.slurm squeue -u gm7719 tail -f rsl.error.0000 ls wrfout_d01_* wc -l ls wrfout_d02_* wc -l
16. For the analysis the scripts are in /scratch/gpfs/RVNE/gabrielm/analysis/ (WRF_analysis_Hunter_tiger3.py for pmin and vmax, track_wrf_tiger3.py for the track), with the wrf_post env where netCDF4 is pinned to 1.6.5 because wrf-python breaks with 1.7. Adding a run is one entry in the RUNS list. Two things not to forget there : the raw min of PSFC (surface pressure) is the Hispaniola mountains and not the storm, so the terrain has to be masked first, and the max wind has to be read on d02.	salloc --nodes=1 --ntasks=1 --mem=32G --time=01:00:00 module load anaconda3/2024.6 source ~/.bashrc conda activate wrf_post cd /scratch/gpfs/RVNE/gabrielm/analysis && python WRF_analysis_Hunter_tiger3.py

Version of conda used : anaconda3/2024.6

What you need for the two conda environments:

Both live in my home in /home/gm7719/.conda/envs/, so you can always look here if you have a doubt. What I actually asked for is short, everything else that shows up in conda list came in as a dependency (I never touched those).

wps_prep is only used to write the intermediate files : python 3.11, numpy, scipy, xarray, and pywintex 2.0.7 from pip (there is no conda package for it, and it cannot be installed on Windows anyway, the package ships example files with a colon in the name, this is why this part is done on the cluster). make_intermediate.py only really needs numpy and pywintex, the rest is leftovers from things I tried at some point, so do not worry if your versions differ.

wrf_post is only used for the analysis scripts : python 3.11, numpy, scipy, pandas, xarray, matplotlib, cartopy, metpy, cf_grib, wrf-python and geocat-viz.

Three pins matter there, and they are the only ones :

numpy < 2, because wrf-python 1.4.0 is not compatible with numpy 2. I ended up with 1.26.4. This is the whole reason why there are two separate environments and not one, wps_prep runs on numpy 2.4.

netcdf4 == 1.6.5, from pip and installed last so that it overwrites the conda one. With netcdf4 >= 1.7 wrf.getvar crashes with a pickling error (it took me a while to find that one).

setuptools < 81, newer versions drop what wrf-python still imports.

The exact versions I ran with, if you ever need to compare : python 3.11.15, numpy 1.26.4, wrf-python 1.4.0, netcdf4 1.6.5, matplotlib 3.11.1, cartopy 0.25.0.

One last thing, conda is not in the PATH by default, neither on the compute nodes nor on the login node. You have to do module load anaconda3/2024.6 then source ~/.bashrc before any conda command, and this has to be redone on every new allocation

About data availability:

Where the input files of build_npz_hybrid.py come from

These are the four paths at the top of the script that have to point somewhere real. Everything else is computation. I write here where they come from in case my Downloads folder disappears.

1. PARENT, the HAFS files

The HAFS-A retrospective forecasts for Fiona, they are the atmospheric background of the whole simulation. They come from the NOAA open bucket on AWS, no account needed : s3://noaa-nws-hafs-pds/hfsa_retro/20220920/06/.

How to read the name 07I.2022092006.hfsa.parent.atm.f009.grb2 : 07I is the storm (Fiona), 2022092006 is the cycle (initialized the 20th at 06 UTC), hfsa is the model, parent is the outer domain at 6 km (the inner nest is finer but too small for our WRF domains), atm is the atmosphere product, f009 is the lead time in hours. So f009 of the 06 UTC cycle is valid at 15 UTC, f012 at 18 UTC, and so on. That is the date written next to each file in FILES, and the two must stay consistent, otherwise you write an intermediate file with a wrong date stamp and metgrid interpolates the wrong field without complaining

Each file is around 700 Mo, so 6 Go for the eight. This is why this step is done on my PC where I already had them from the PINN work.

2. SOIL_GRIB, the GFS soil file

HAFS does not give the four soil layers that the Noah scheme needs (soil temperature and moisture at 0-10, 10-40, 40-100, 100-200 cm) and real.exe refuses to run without them, so we take them from GFS. This is the only thing this file is used for, plus the skin temperature.

It comes from the GFS archive (NOMADS, or s3://noaa-gfs-bdp-pds/), 00 UTC cycle of the 20th, analysis. It is only 3 Mo because I subsetting it to the 8 fields we need instead of downloading the full GRIB. The 1(h hours difference is considered negligible for the soil here.

3. MODEL_PATH: the PINN checkpoint

Which PINN vortex are we using. It is produced on Neuronic and the name is built automatically : fiona, 3D, with T and q in the outputs, sequential training, run of the 13th of August 2026 at 02:16:03, checkpoint at the end of phase 1. They stay on Neuronic and I copy to the PC only the ones I want to inject.

The file contains the weights and the normalization (t_min, t_max, etc). The script rebuilds the architecture by hand before loading the state dict, so if the network changes (width, depth, sigma of the Fourier features), **the class inside build_npz_hybrid.py has to be changed the same way**, if not it crashes, or worse it loads silently into a wrong architecture

4. OUT_NPZ, the output

What the script produces, and the only file that has to go to tiger3. Eight records, the first one already blended, in the flat form that make_intermediate.py is waiting for. Around 1 Go. The letter at the end (g here) is the run identifier and it must match the directory fiona_wrf_pinn15g, since this is what the sed of step 4 is looking for.

About the paths themselves : they are absolute Windows paths, which is not beautiful but it was the fastest since everything was on my laptop. If you move the script to tiger3, these four lines are the only thing to change, but you will also need the GRIB files there (6 Go) and the checkpoint.